GDP 301 Systems programming and cross-platform development
Week 2 – Cross-platform toolchains, continuous integration and automated build generation

1. Introduction

In these tutorials, we will explore what is a CI/CD pipeline and how to setup a first automation environment for Unreal Engine 5 builds.

We will be using Jenkins for this process. Jenkins is an open-source automation server widely used for Continuous Integration/Continuous Deployment pipelines, and automates various stages of software development including building, testing and deployment.

Although you can install Jenkins on your own device, we have created an IT managed server here for you to use.

You are going to learn how to setup an automated build pipeline using Jenkins, how to interact with the UAT (Unreal Automation Tool) and how to create a Jenkins script that combines all stages needed To build your project automatically.

You will gain a foundational understanding of both CI/CD pipelines and how to integrate these tools in your own projects.

Important:

- **This tutorial runs in a Windows machine, in case you use a Linux machine, you may want to convert the CMD commands inside the stages to the ones from the Linux CLI**

2. Objectives

Learning outcomes for the unit On successful completion of this unit students will be able to:
Demonstrate broad technical and theoretical knowledge of the requirements and constraints of cross-platform game development including GUI design, data and resource management, profiling, mobile optimisation and shaders.
Use cognitive, technical and creative skills to produce innovative design concepts and implement continuous integration for a cross-platform game project.
Communicate effectively in written formats.
Apply initiative and judgement in the production and deployment of cross-platform games to multiple platforms.

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

3. Background

In the previous lecture, we explored the fundamental principles of how to plan for multiplatform games.

We discussed the basics of how to start planning and making assertive technical decisions for the development of a cross-platform game, understanding the pros and cons of each platform.

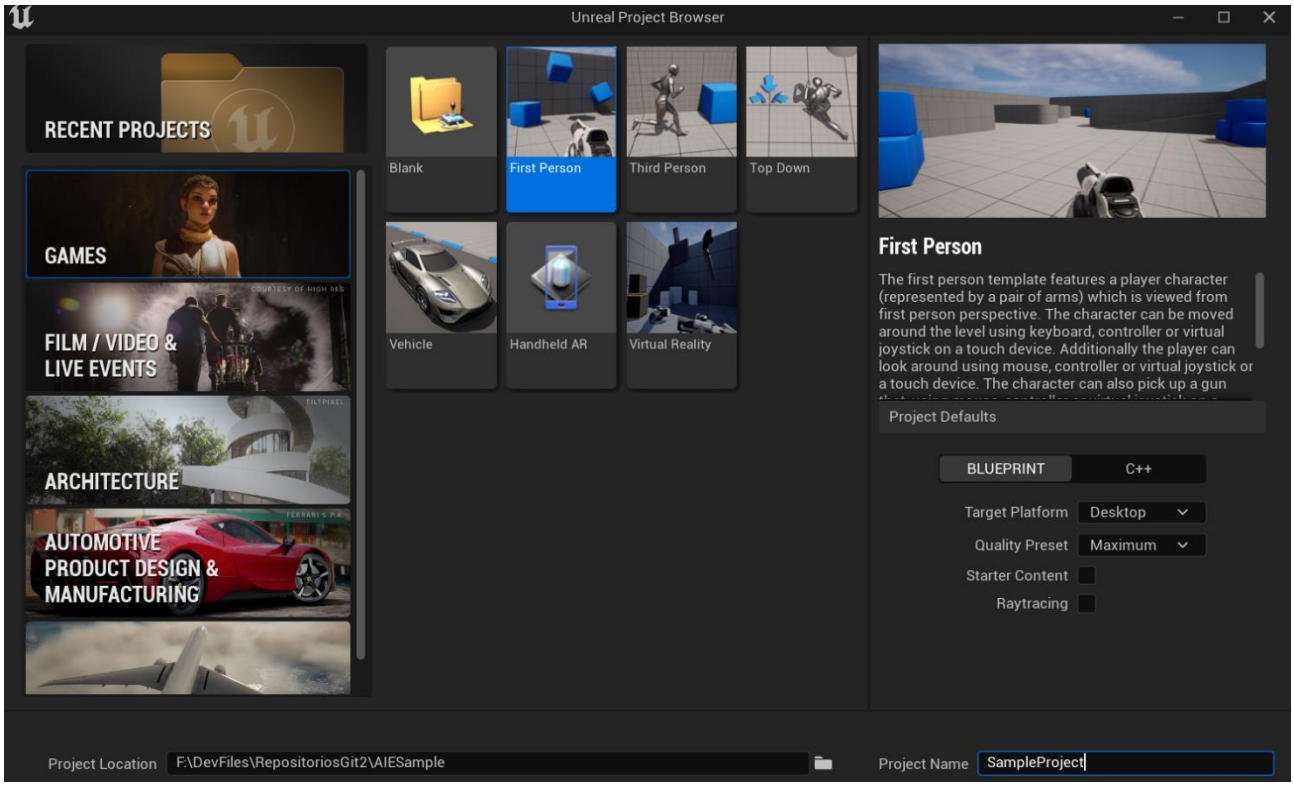
These tutorials should give you basic's on how to use the UAT, setup a Jenkins pipeline and automate Unreal Engine processes within it.

4. Tutorial Tasks

A Jenkins server has been provided by the Institute, the URL is **10.100.10.140:8080** (make sure your computer is logged into the Institute VPN if you are doing this from home)

You will need an Unreal project to test with. Create a project based on the FPS template using Blueprint, as the image shows.

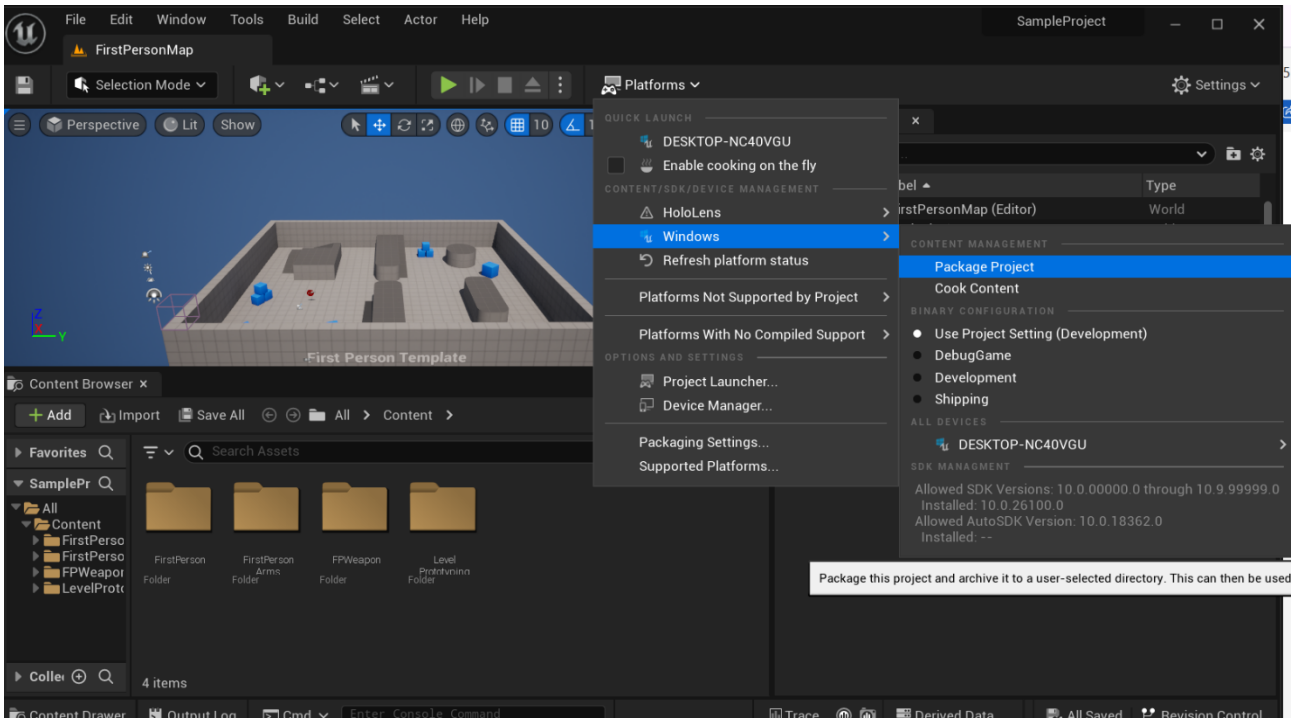
You may use another UE project if you want but a bare-minimum project is best for this tutorial.



After creating the project, make sure it builds correctly at least once by clicking on Platforms->Windows->Package project:

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F



Now, let's get it started!

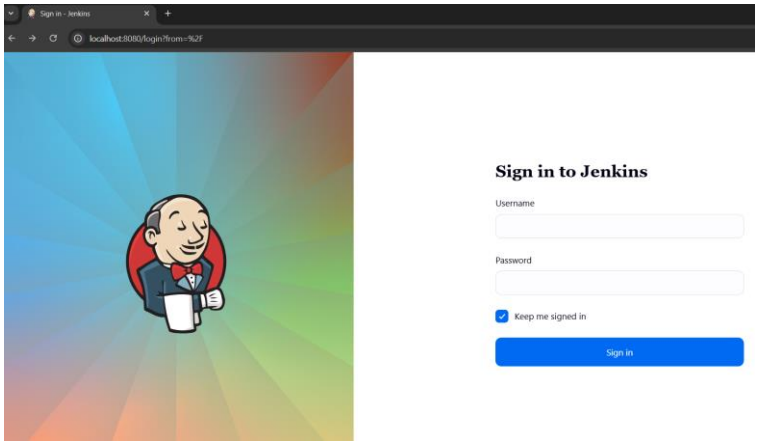
Below is 1 tutorial that will take you through the concepts of setting up a CI/CD pipeline with Jenkins and then integrating it with Unreal Engine 5 to generate automated builds.

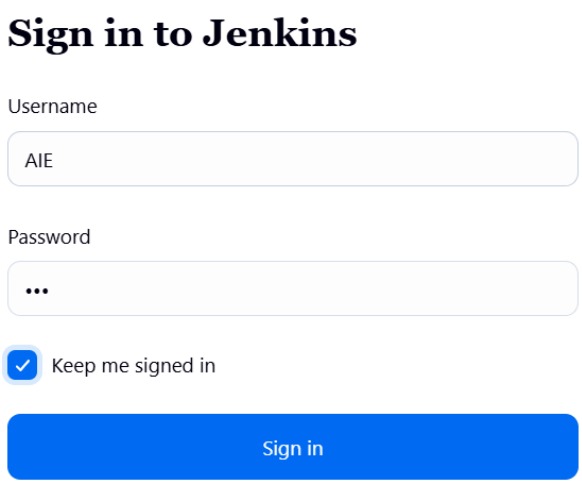
1.1 Jenkins Build Automation - Tutorial

Within this tutorial the goal is to setup a Jenkins environment to create a simple CI/CD that tags and build our Unreal Engine 5 project using manual triggers.

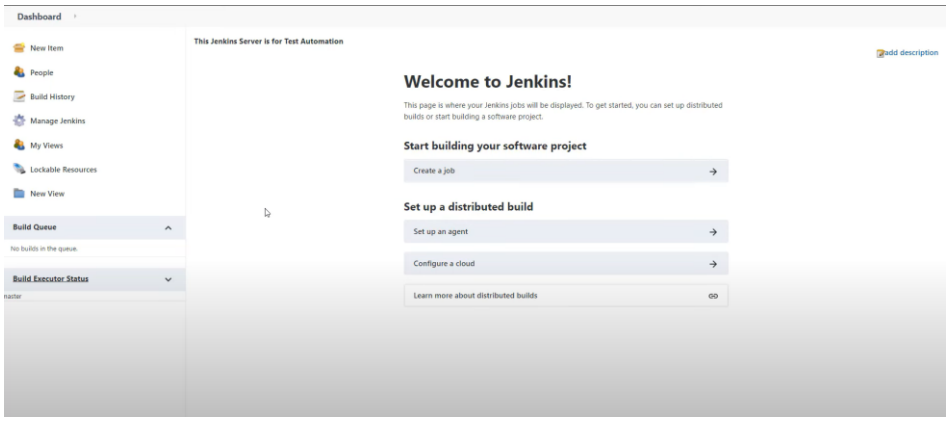
Solution:

1.1.1 Starting up with Jenkins





After that, you will see the Jenkins initial page:

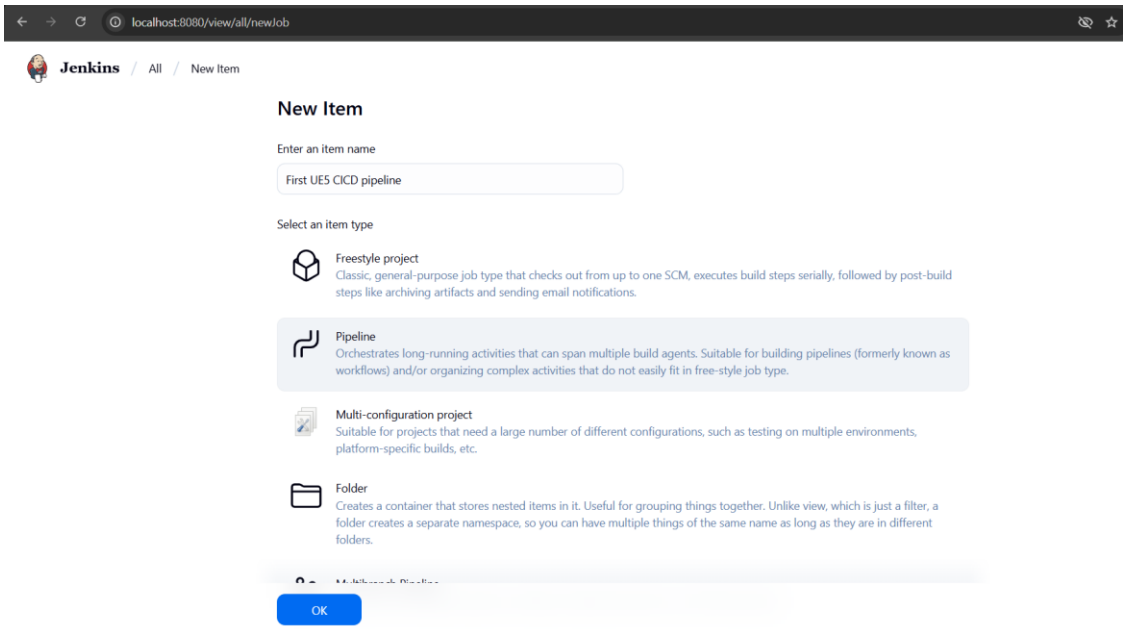


Jenkins is all set up for use, now let's select **'Create a job'**.

On the next page, let's mark that we want to create a **'pipeline'** and name it for something similar to **'First UE5 CICD pipeline'**:

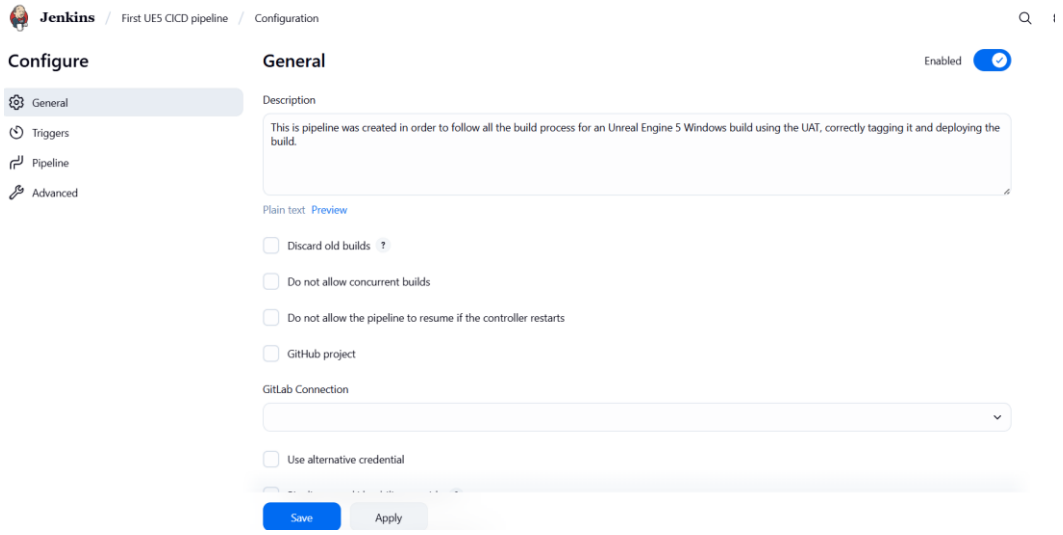
AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F



Inside the description page, we can write **'This is pipeline was created in order to follow all the build process for an Unreal Engine 5 Windows build using the UAT, correctly tagging and deploying the build.'**

The other fields can be left in blank, and then we should scroll down:

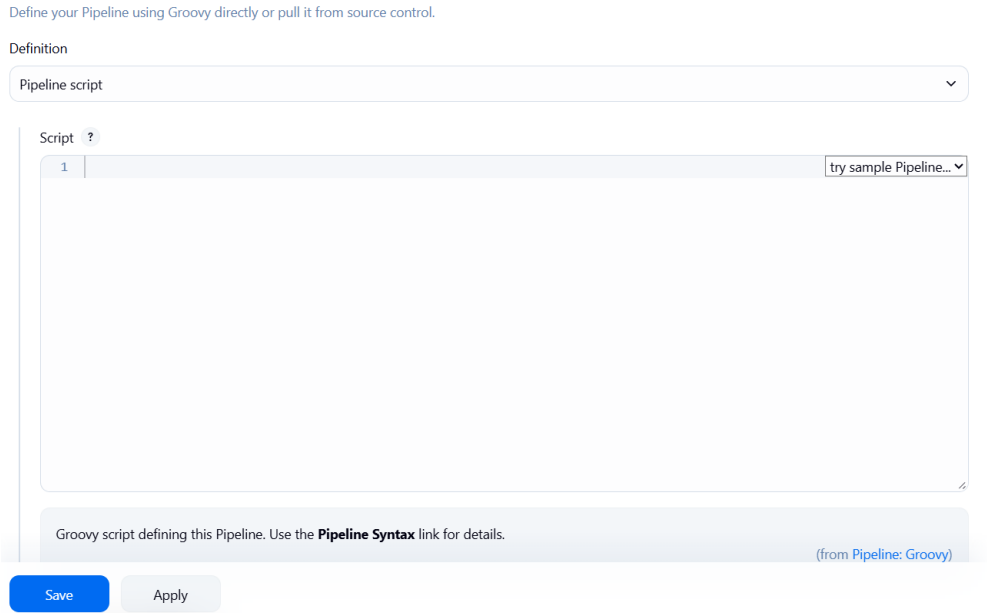


1.1.2 Starting up with the pipeline

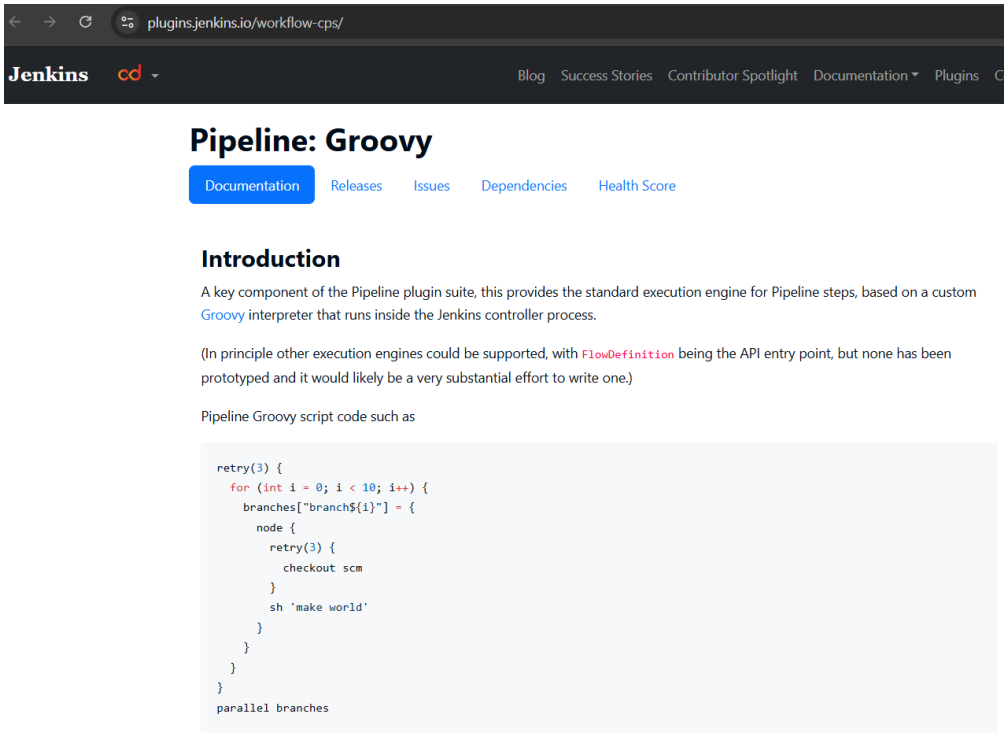
Arriving in the 'Pipeline' section, it is possible to see an area where we can create our scripts. There is an '?' button besides the 'Script' text. You can click it and then go to this page https://plugins.jenkins.io/workflow-cps/ that explains about that script's syntax – Groovy is an interpreter that runs inside the Jenkins controller process:

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F



Pipeline: Groovy link:



To start our process, we want to create one build for Windows, so in this case, we first want to get the path to 3 files that we will use (you can change the **UE_5.2** for your installed Unreal version):

- The UAT file

UAT_PATH = "C:\EpicGames\UE_5.4\Engine\Build\BatchFiles\RunUAT.bat" PROJECT_PATH = "C:\data\projects\JenkinsTest\JenkinsTest\JenkinsTest.uproject" BUILD_OUTPUT = "C:\data\projects\JenkinsTest\JenkinsBuild\"

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

- The .uproject file of the project we want to build (in my case F:\\DevFiles\\RepositoriosGit2\\AIESample\\SampleProject\\SampleProject.uproject)

"C:\\data\\projects\\<your project name>\\<yourprojectname>.uproject

- The folder where we are going to deploy and save our artifacts (in this case, you can create a folder anywhere that has read/write permissions, in my case it will be F:\\DevFiles\\RepositoriosGit2\\AIESample\\JenkinsBuild)

"C:\\data\\projects\\JenkinsTest\\JenkinsBuild\\"

NB. Be careful to include **double ** where indicated!

So that, the first thing we want to create on our script is the definition of our pipeline, the agent and the path for those files:

Script ?

```
pipeline {
    agent any

    environment {
        UAT_PATH = "C:\\Program Files\\Epic Games\\UE_5.2\\Engine\\Build\\BatchFiles\\RunUAT.bat"
        PROJECT_PATH = "F:\\DevFiles\\RepositoriosGit2\\AIESample\\SampleProject\\SampleProject.uproject"
        BUILD_OUTPUT = "F:\\DevFiles\\RepositoriosGit2\\AIESample\\JenkinsBuild"
    }
}
```

And then we click Apply

Now, let's create our tab of Stages and our first stage -> the Build process:

```

    stages {

        stage('Build') {
            steps {
                echo "Compiling C++ source code..."
                bat """
                call "%UAT_PATH%" BuildCookRun ^
                 -project="%PROJECT_PATH%" ^
                 -noP4 ^
                 -platform=Win64 ^
                 -clientconfig=Development ^
                 -build ^
                 -skipcook -skipstage -skippak -skiparchive
                """
```


AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

And then, when everything is done, we want the Jenkins to give us a quick feedback if it succeded or not:

```
    post {
        success {
            echo "Unreal Engine build completed successfully!"
        }
        failure {
            echo "Build failed. Check the logs under the Jenkins console output."
        }
    }
```

It is important to notice that we are using Windows commands, since it is running on my local machine. Most automation tools use Linux containers or virtual machines because it's smaller and faster than other OS for some tasks, and in that case you would use Linux commands.

You can copy and paste the following code:

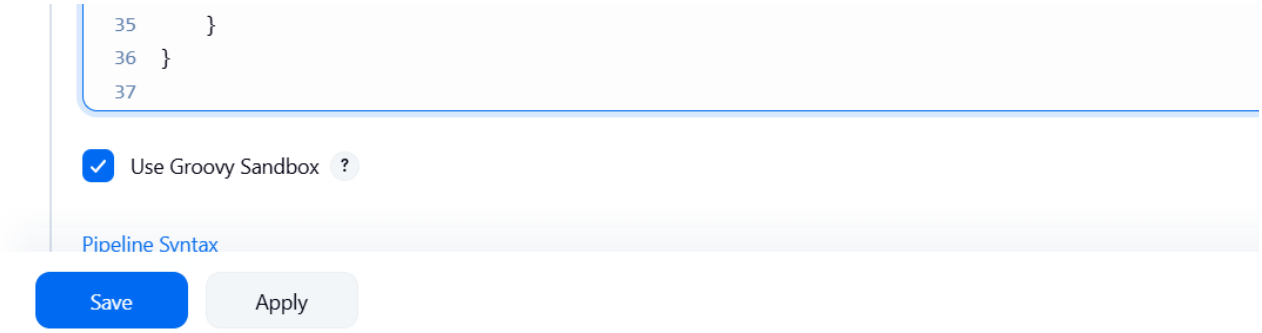
Code

```
pipeline { agent any

environment {
    UAT_PATH = "C:\\Program Files\\Epic
Games\\UE_5.2\\Engine\\Build\\BatchFiles\\RunUAT.bat"
    PROJECT_PATH =
"F:\\DevFiles\\RepositoriosGit2\\AIESample\\SampleProject\\SampleProject.uproje
ct"
    BUILD_OUTPUT = "F:\\DevFiles\\RepositoriosGit2\\AIESample\\JenkinsBuild"
}

stages {

    stage('Build') {
        steps {
            echo "Compiling C++ source code..."
            bat """
            call "%UAT_PATH%" BuildCookRun ^
             -project="%PROJECT_PATH%" ^
             -noP4 ^
             -platform=Win64 ^
             -clientconfig=Development ^
             -build ^
             -skipcook -skipstage -skippak -skiparchive
            """
        }
    }
}

post {
    success {
        echo "Unreal Engine build completed successfully!"
    }
```


AIE Institute Limited | ABN 91 624 067 536 Higher Education Provider PRV14333 | CRICOS 03986F

```
    failure {
        echo "Build failed. Check the logs under the Jenkins console output."
    }
}
```

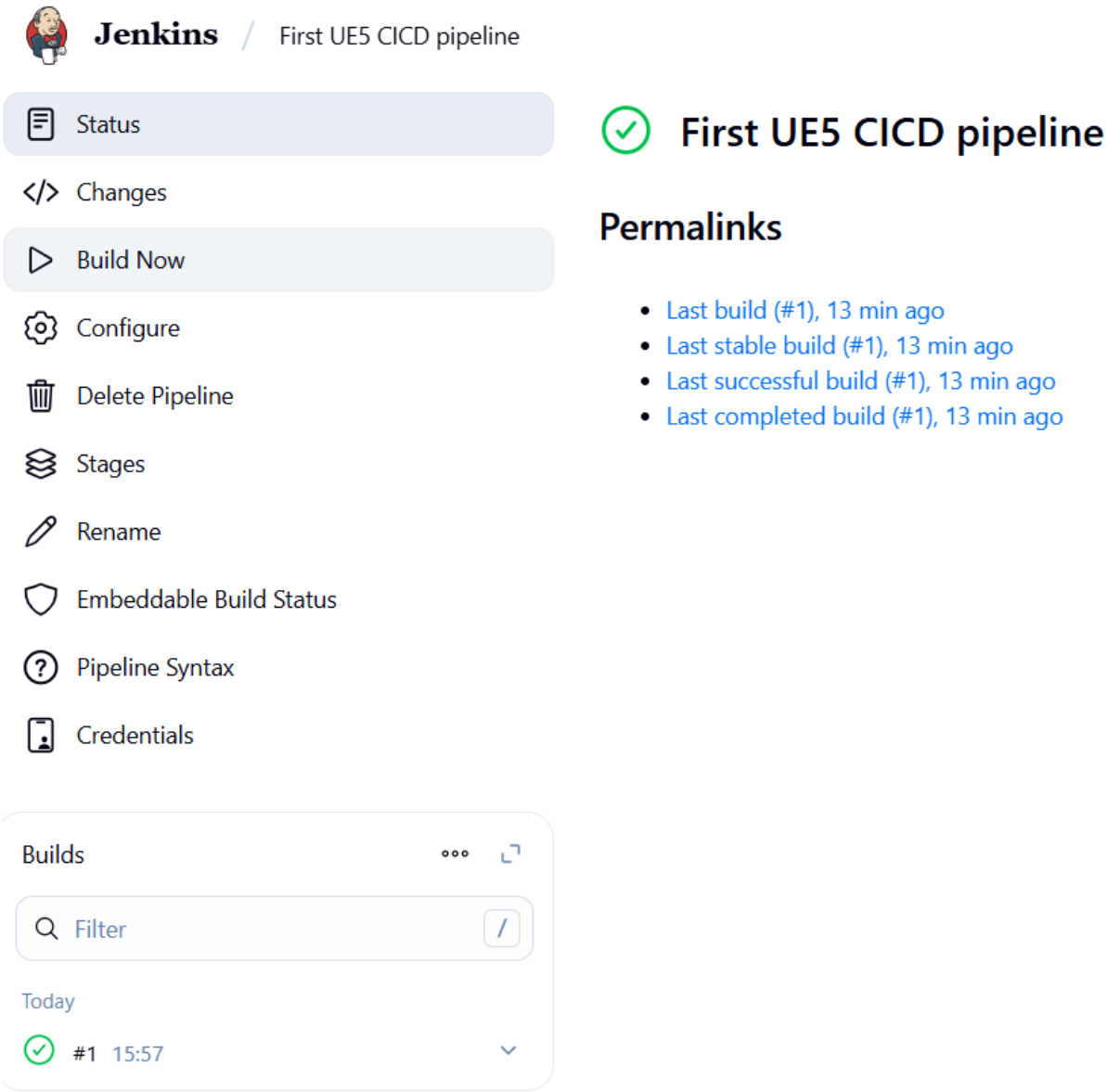
Now, let's keep the **Use Groovy Sandbox** checked, click **apply** and then click **Save**:



On the next screen, we can select Build Now and see the process:

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

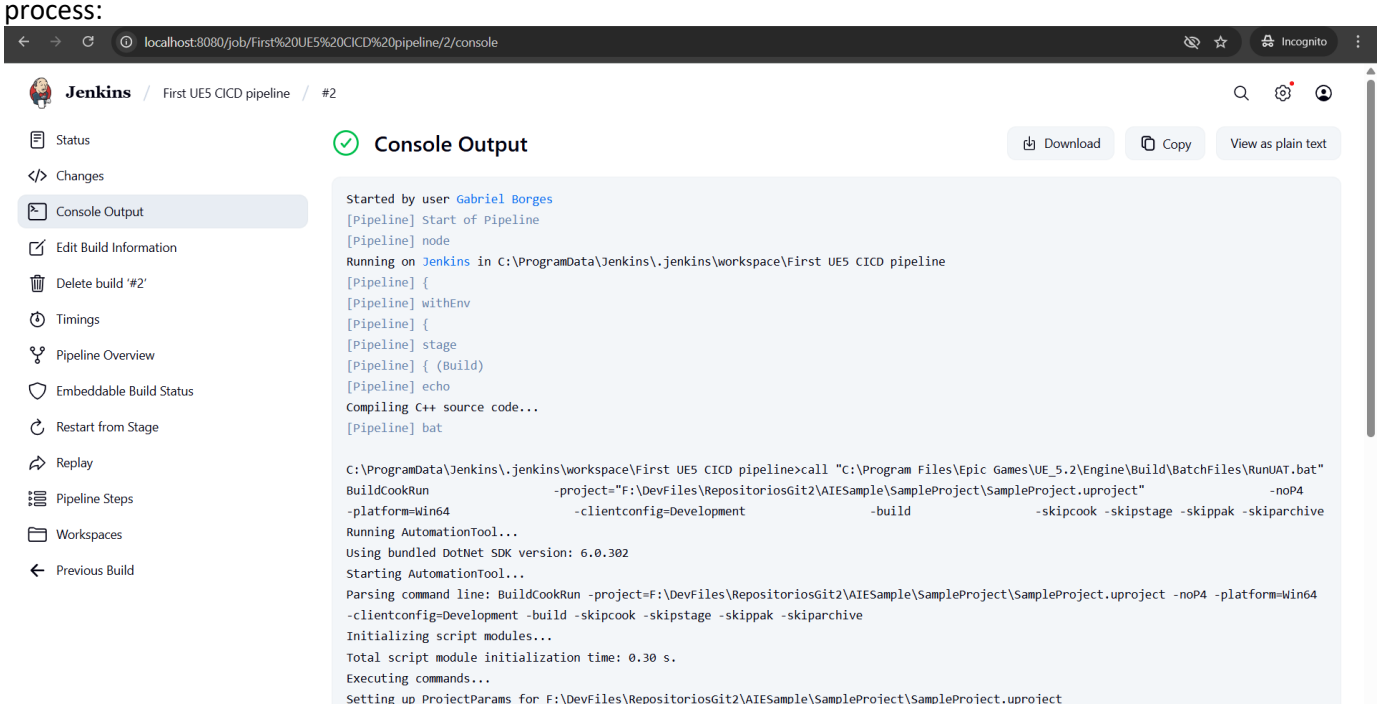


If your build does not appear, press F5, and check on the left down corner.

AIE Institute Limited | ABN 91 624 067 536

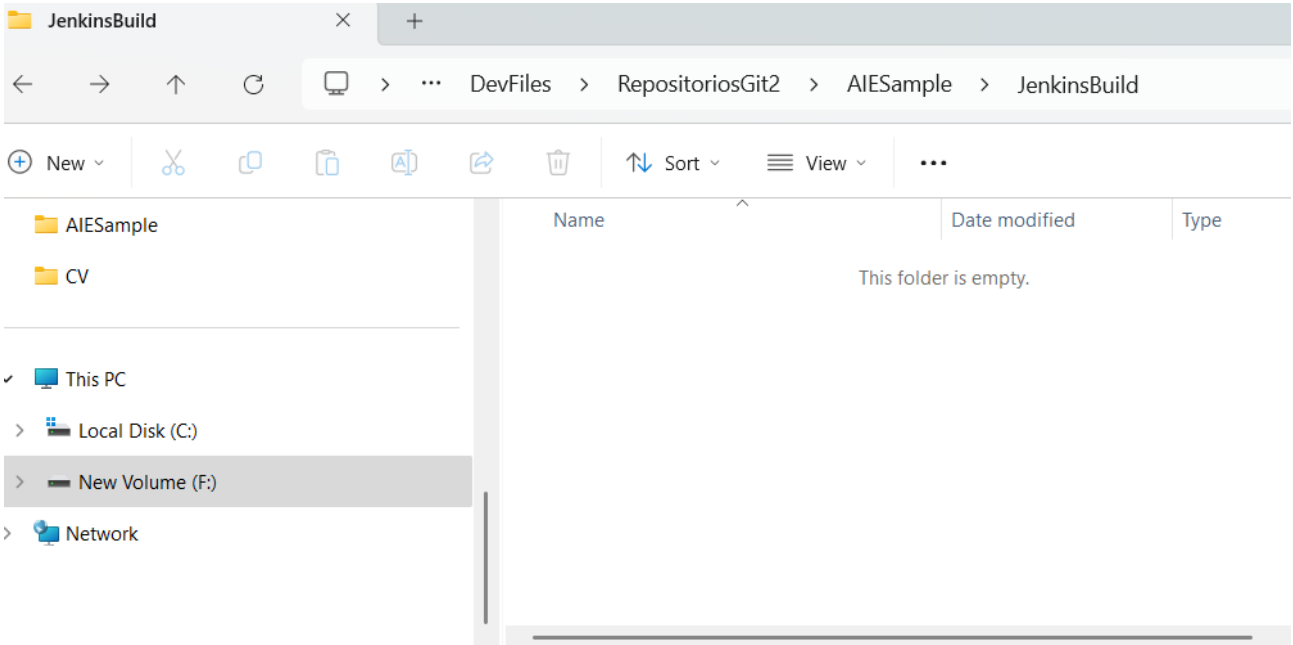
Higher Education Provider PRV14333 | CRICOS 03986F

Select your build, and then select Console Output to understand Everything that happened during this process:



At the first lines, you are able to see that Jenkins is creating and agent and running your job inside a folder in your PC. Also, in the last lines, you should be able to see that the build succeeded.

Great! Now, if we check our folder, you will see that there is no build there. This is because the first step of building is just to compile, and not to generate the executable file.



So, our next step of the build will be the cook step. Click on the **Jenkins Logo** at the top left corner, select your **Pipeline**, then select **Configure** to see our pipeline script again. Add this stage into your Jenkins:

Code:

```
stage('Cook') {
```


AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

```
    steps {
      echo "Cooking content..."
      bat """
      call "%UAT_PATH%" BuildCookRun ^
       -project="%PROJECT_PATH%" ^
       -noP4 ^
       -platform=Win64 ^
       -clientconfig=Development ^
       -cook ^
       -skipbuild -skipstage -skippak -skiparchive
      """
    }
  }
```

If you check the console output again, you can see a metadata with all the files being converted and compressed, and that the pipeline succeeded.

Now, let's finish all the other stages (stage, package and deploy):

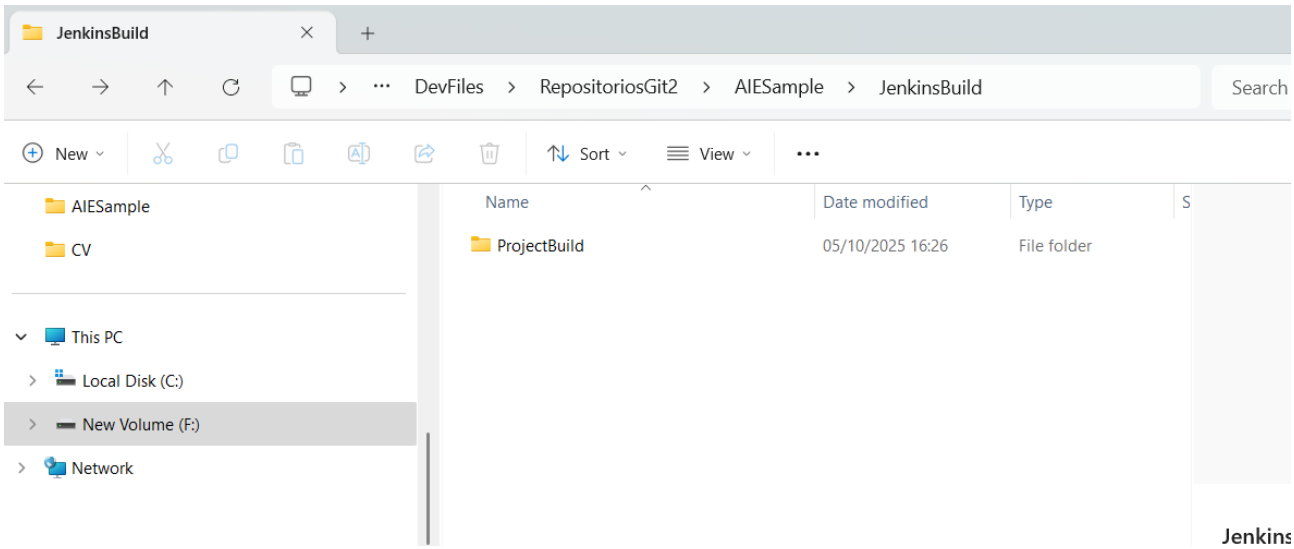
Code:

```
stage('Stage') {
     steps {
         echo "Staging cooked content..."
         bat """
         call "%UAT_PATH%" BuildCookRun ^
          -project="%PROJECT_PATH%" ^
          -noP4 ^
          -platform=Win64 ^
          -clientconfig=Development ^
          -stage ^
          -skipbuild -skipcook -skippak -skiparchive
         """
     }
 }

 stage('Pak') {
     steps {
         echo " Creating .pak files..."
         bat """
         call "%UAT_PATH%" BuildCookRun ^
          -project="%PROJECT_PATH%" ^
          -noP4 ^
```


AIE Institute Limited | ABN 91 624 067 536 Higher Education Provider PRV14333 | CRICOS 03986F

```
            -platform=Win64 ^
            -clientconfig=Development ^
            -pak ^
            -skipbuild -skipcook -skipstage -skiparchive
           """
        }
    }

    stage('Archive') {
        steps {
            echo "Archiving packaged build..."
            bat """
            call "%UAT_PATH%" BuildCookRun ^
             -project="%PROJECT_PATH%" ^
             -noP4 ^
             -platform=Win64 ^
             -clientconfig=Development ^
             -archive ^
             -archivedirectory="%BUILD_OUTPUT%\\ProjectBuild" ^
             -skipbuild -skipcook -skipstage -skippak
            """
        }
    }
```

After everything finish to run, we do have our first automated build!



You can try to play the executable file to see if it works now.

1.1.3 Polishing and adjusting the pipeline

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

Now, for the next step, lets tag the build version in a way that the team will be able to easily identify. We are going to use a traditional setup that will contain the Platform, Number of the build, Date and Time it was generated:

Code to be put as our first stage:

```
stage('Initialize Build Tag') { steps { script { def timestamp = new Date().format('yyyyMMdd_HHmm') env.BUILD_TAG = "${env.BUILD_NUMBER}_${timestamp}" echo "Build tag set to: ${env.BUILD_TAG}" } } }
```

Code to be put in the **Archive** stage. This line of code should replace the line that define the file output path:

```
        stage('Archive') {
            steps {
                echo "Archiving packaged build..."
                bat """
                call "%UAT_PATH%" BuildCookRun ^
                 -project="%PROJECT_PATH%" ^
                 -noP4 ^
                 -platform=Win64 ^
                 -clientconfig=Development ^
                 -archive ^
                 -archivedirectory="%BUILD_OUTPUT%\\WindowsBuild_${env.BUILD_TAG}" ^
                 -skipbuild -skipcook -skipstage -skippak
                """
```

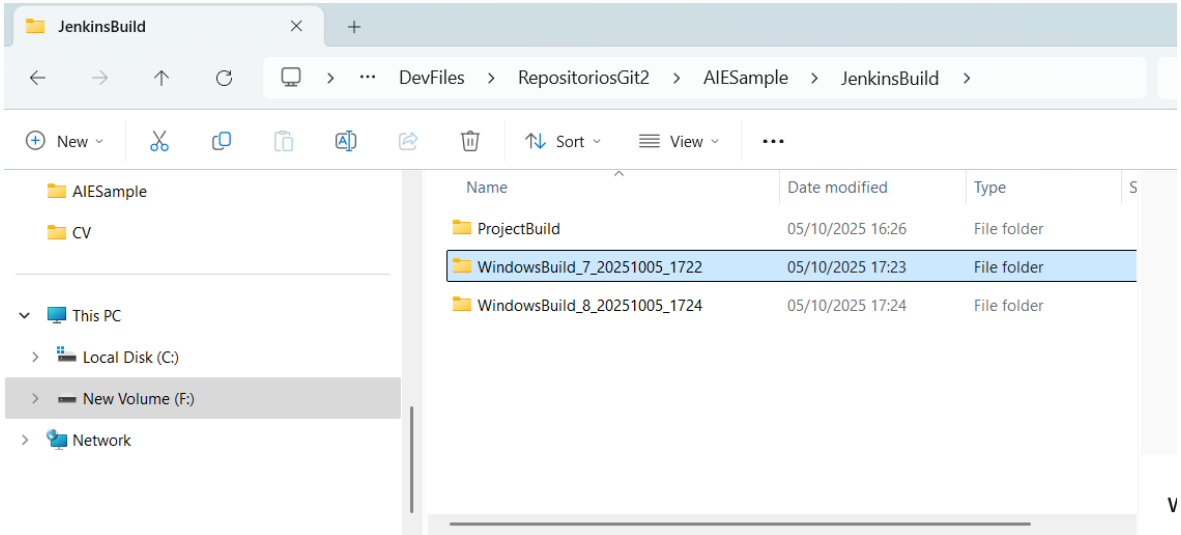
Code:

```
                -archivedirectory="%BUILD_OUTPUT%\\WindowsBuild_${env.BUILD_TAG}" ^
```

Now, when we build again, we will see that our build is no longer overwritten, and also that now we do have a build history:

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F



For the teacher: -if you want, you can make a change in your scene, save, trigger the build and show it is not being saved for the Jenkins build.-

Our last step will be to integrate all those stages into one single method. Multiple stages are interesting to debug every process of the build and understand how it's being done, but UE5 expects the process to be done sequentially (within the same BuildCookRun command).

In our current process, the output files from the Compiled, Cooked and Packed stages are living only inside the Jenkins machine as artifacts, and we are not passing it forwards. It means that in the last stage, where we Archive, we use the outdate files in our current machines (the unreal project path).

For that reason, we are going to delete some stages and implement one as it follows:

Code:

```
stages {

    stage('Initialize Build Tag') {
        steps {
            script {
                def timestamp = new Date().format('yyyyMMdd_HHmm')
                env.BUILD_TAG = "${env.BUILD_NUMBER}_${timestamp}"
                echo "Build tag set to: ${env.BUILD_TAG}"
            }
        }
    }

    stage('Build') {
        steps {
```


AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

```
            echo "Starting Unreal Engine BuildCookRun..."
            bat """
            call "%UAT_PATH%" BuildCookRun ^
             -project="%PROJECT_PATH%" ^
             -noP4 ^
             -platform=Win64 ^
             -clientconfig=Development ^
             -build ^
             -cook ^
             -stage ^
             -pak ^
             -archive ^
             -
archivedirectory="%BUILD_OUTPUT%\\WindowsBuild_${env.BUILD_TAG}" ^
             -cookdir=/Game ^
             -unversionedcookedcontent ^
             -clean
            """
        }
    }
}
```

Now, you are able to do one last change inside the scene of your project, like moving the first gun to another position or duplicating the cubes we already have in the scene. Now, after saving it and trigger another build, you should see all these changes applied!

5. Revision History

Approval authority	Contact person	Revision due date	Revision date	Approved date	Version no.	Revision description
Academic Board	Chair					
Drafting details						

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F

Prepared by	Preparation date	Revised by	Revision date		Draft no.	Revision description
GB	05/10/2025				V0.01	Preparation of new document
		Enter initials of person(s) making the revisions	Enter revision date/ time period of revisions		V0.02 etc	Enter a brief description of the amendments made.

AIE Institute Limited | ABN 91 624 067 536

Higher Education Provider PRV14333 | CRICOS 03986F